

车载机器人样机小智实现方案

基于当前小智开源代码架构，结合车联网和 NSP 后台接口，建议按以下方案进行功能扩展：

1. 硬件选型

选择成熟的传感器模块，如 MPU6050（加速度计和陀螺仪）等，确保与 [小智实战派](#) 主板兼容。主板支持 I2C [淘宝链接](#)。

1.1 硬件接线框图

```
@startuml
skinparam componentStyle rectangle
component "MPU6050\n(I2C)" as MPU6050
component "实战派\n(SparkBot)" as 实战派
MPU6050 -[hidden]down-> 实战派 : VCC, GND
MPU6050 -[hidden]down-> 实战派 : SDA, SCL
@enduml
```

2. 软件方案

2.1. 驾驶行为分析

- 增加加速度传感器、陀螺仪等硬件模块，采集车辆加速、陡坡等数据。
- 通过 I2C 读取传感器数据，定时存储和分析。
- 在主循环中统计加速次数、坡度变化，设定阈值，超限时语音提醒。
- 在 SparkBot 主板类或 Application 类中增加传感器初始化和数据处理逻辑。

是否有现成的基于I2C控制的MPU6050的库

ESP32 平台上常用的 MPU6050 通信方式为 I2C（而非串口），但也有部分模块支持 UART 转 I2C 方案。推荐如下：

- **I2C 方式（主流，推荐）：**
 - [esp-idf-mpu6050](#)（适用于 ESP-IDF，I2C 通信）
 - [esp32-mpu6050-driver](#)（C 语言，I2C 通信）
 - Arduino生态下也有大量成熟库，移植到 ESP-IDF 较为容易。
- **集成建议：**
 1. 优先采用 I2C 方式，直接用 ESP32 的 I2C 驱动和上述开源库。
 2. 代码集成时，建议将传感器采集、数据解析、异常处理封装为独立类，主循环定时读取并分析。
- **参考代码片段（I2C, ESP-IDF）：**

```

#include "MPU6050.h"
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

MPU6050 mpu;

void mpu6050_task(void *pvParameters) {
    if (!mpu.initialize()) {
        printf("MPU6050 init failed!\n");
        vTaskDelete(NULL);
    }
    printf("MPU6050 init success!\n");
    while (1) {
        int16_t ax, ay, az, gx, gy, gz;
        if (mpu.testConnection()) {
            mpu.getAcceleration(&ax, &ay, &az);
            mpu.getRotation(&gx, &gy, &gz);
            printf("Accel: X=%d Y=%d Z=%d | Gyro: X=%d Y=%d Z=%d\n", ax, ay,
az, gx, gy, gz);
            // 简单行为分析示例
            if (abs(ax) > 16000 || abs(ay) > 16000) {
                printf("警告：检测到剧烈加速或急转弯！\n");
            }
        } else {
            printf("MPU6050 连接异常！\n");
        }
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

void app_main() {
    xTaskCreate(mpu6050_task, "mpu6050_task", 4096, NULL, 5, NULL);
}

```

2.2. 车辆数据获取（车联网后台）

- 根据车联网后台文档，集成 HTTP/REST 或 WebSocket 客户端。
- 定时或按需请求 `/vehicle/status` 等接口，获取里程、速度、SOC 等数据。
- 解析 JSON 响应，语音播报或显示屏展示。
- 在 `Application` 类中增加车辆数据请求和处理函数。

2.3. 车辆控制（远程开门等）

- 集成车联网控制接口，如 `/vehicle/control`。
- 用户语音下达控制指令，设备通过 API 发送控制请求。
- 关键操作需二次确认，防止误操作。
- 在语音指令解析模块和 `Application` 类中增加控制逻辑。

2.4. 保养信息查询（NSP后台）

- 集成 NSP 后台接口，如 `/vehicle/maintenance`。
- 定期或按需请求保养信息，解析剩余里程等数据。
- 将保养信息通过语音或屏幕展示给用户。
- 在 `Application` 类中增加保养信息请求和处理函数。

2.5. 通用建议

- 所有接口建议采用异步请求，防止阻塞主循环。
- 增加错误处理和重试机制，提升稳定性。
- 所有新功能使用python基于mcp协议测试确保稳定可靠加速开发。

2.6. 使用mcp协议的好处

2.7. 车联网接口MCP工具参考实现

以下所有车联网相关接口均建议通过 MCP Tool 机制集成，便于统一管理和云端调用。

2.7.1 登录接口（获取token）

- MCP工具名：`car.login`
- 工具描述：登录车联网平台，获取token
- 输入参数：用户名、密码、登录类型、系统标识
- 返回：token、tokenIssuer
- 参考实现：

```
mcp_server.AddTool("car.login", "车联网平台登录", PropertyList({
    Property("loginName", kPropertyTypeString),
    Property("loginPwd", kPropertyTypeString),
    Property("loginType", kPropertyTypeString, "1"),
    Property("providerSys", kPropertyTypeString, "electric")
}), [this](const PropertyList& properties) -> ReturnValue {
    // 构造JSON请求体
    cJSON* req = cJSON_CreateObject();
    cJSON_AddStringToObject(req, "providerSys",
properties["providerSys"].value<std::string>().c_str());
    cJSON_AddStringToObject(req, "loginName",
properties["loginName"].value<std::string>().c_str());
    cJSON_AddStringToObject(req, "loginPwd",
properties["loginPwd"].value<std::string>().c_str());
    cJSON_AddStringToObject(req, "loginType",
properties["loginType"].value<std::string>().c_str());
    std::string body = cJSON_PrintUnformatted(req);
    cJSON_Delete(req);
    // 发送HTTP POST
    std::string url =
"https://gateway.wuzheng.com.cn/common/user/user/common_login";
    std::string response = HttpPost(url, body, {}); // HttpPost为自定义HTTP工具函数
    return response;
});
```

2.7.2 车辆实时数据接口

- MCP工具名：`car.get_realtime_data`
- 工具描述：获取车辆实时数据
- 输入参数：`token`、`tokenIssuer`、`vin`
- 返回：车辆实时数据JSON
- 参考实现：

```
mcp_server.AddTool("car.get_realtime_data", "获取车辆实时数据", PropertyList({
    Property("token", kPropertyTypeString),
    Property("tokenIssuer", kPropertyTypeString),
    Property("vin", kPropertyTypeString)
}), [this](const PropertyList& properties) -> ReturnValue {
    std::string url =
"https://gateway.wuzheng.com.cn/vehicle/data/partner/api/ai/get/real/times?
sign=sign&timestamp=" + std::to_string(time(NULL));
    cJSON* req = cJSON_CreateObject();
    cJSON_AddStringToObject(req, "vin", properties["vin"].value<std::string>
()).c_str());
    std::string body = cJSON_PrintUnformatted(req);
    cJSON_Delete(req);
    std::map<std::string, std::string> headers = {
        {"Authorization", properties["token"].value<std::string>()},
        {"X-Token-Issuer", properties["tokenIssuer"].value<std::string>()},
        {"X-Request-Id", "1"}
    };
    std::string response = HttpPost(url, body, headers);
    return response;
});
```

2.7.3 车辆充电统计接口

- MCP工具名：`car.get_charge_statistics`
- 工具描述：获取车辆充电统计数据
- 输入参数：`token`、`tokenIssuer`、`vin`、`dateBegin`、`dateEnd`
- 返回：充电统计JSON
- 参考实现：

```
mcp_server.AddTool("car.get_charge_statistics", "获取车辆充电统计数据",
PropertyList({
    Property("token", kPropertyTypeString),
    Property("tokenIssuer", kPropertyTypeString),
    Property("vin", kPropertyTypeString),
    Property("dateBegin", kPropertyTypeString),
    Property("dateEnd", kPropertyTypeString)
}), [this](const PropertyList& properties) -> ReturnValue {
    std::string url =
"https://gateway.wuzheng.com.cn/vehicle/analysis/bos/chargeStatistics?
sign=sign&timestamp=" + std::to_string(time(NULL));
```

```

    cJSON* req = cJSON_CreateObject();
    cJSON_AddStringToObject(req, "vin", properties["vin"].value<std::string>
    ().c_str());
    cJSON_AddStringToObject(req, "dateBegin",
    properties["dateBegin"].value<std::string>().c_str());
    cJSON_AddStringToObject(req, "dateEnd",
    properties["dateEnd"].value<std::string>().c_str());
    std::string body = cJSON_PrintUnformatted(req);
    cJSON_Delete(req);
    std::map<std::string, std::string> headers = {
        {"Authorization", properties["token"].value<std::string>()},
        {"X-Token-Issuer", properties["tokenIssuer"].value<std::string>()},
        {"X-Request-Id", "1"}
    };
    std::string response = HttpPost(url, body, headers);
    return response;
});

```

2.7.4 车辆车门控制下发接口

- MCP工具名：`car.door_control`
- 工具描述：下发车辆车门控制指令
- 输入参数：token、tokenIssuer、vin、cmdType、parameter（JSON）
- 返回：控制结果JSON
- 参考实现：

```

mcp_server.AddTool("car.door_control", "车辆车门控制", PropertyList({
    Property("token", kPropertyTypeString),
    Property("tokenIssuer", kPropertyTypeString),
    Property("vin", kPropertyTypeString),
    Property("cmdType", kPropertyTypeInteger),
    Property("parameter", kPropertyTypeString)
}), [this](const PropertyList& properties) -> ReturnValue {
    std::string url =
    "https://gateway.wuzheng.com.cn/vehicle/control/bos/issue_terminal_control_command
    ";
    cJSON* req = cJSON_CreateObject();
    cJSON_AddStringToObject(req, "vin", properties["vin"].value<std::string>
    ().c_str());
    cJSON_AddStringToObject(req, "protocolType", "GB32960");
    cJSON_AddNumberToObject(req, "issueTime", (double)time(NULL) * 1000);
    cJSON_AddNumberToObject(req, "cmdType", properties["cmdType"].value<int>());
    cJSON* param = cJSON_Parse(properties["parameter"].value<std::string>
    ().c_str());
    cJSON_AddItemToObject(req, "parameter", param);
    std::string body = cJSON_PrintUnformatted(req);
    cJSON_Delete(req);
    std::map<std::string, std::string> headers = {
        {"Authorization", properties["token"].value<std::string>()},
        {"X-Token-Issuer", properties["tokenIssuer"].value<std::string>()},
        {"X-Request-Id", "1"}
    };

```

```
};
std::string response = HttpPost(url, body, headers);
return response;
});
```

2.7.5 车辆车门控制结果查询接口

- MCP工具名：`car.get_door_control_result`
- 工具描述：查询车门控制结果
- 输入参数：token、tokenIssuer、vin
- 返回：控制结果JSON
- 参考实现：

```
mcp_server.AddTool("car.get_door_control_result", "查询车门控制结果",
PropertyList({
    Property("token", kPropertyTypeString),
    Property("tokenIssuer", kPropertyTypeString),
    Property("vin", kPropertyTypeString)
}), [this](const PropertyList& properties) -> ReturnValue {
    std::string url =
"https://gateway.wuzheng.com.cn/vehicle/control/bos/get_locked_state";
    cJSON* req = cJSON_CreateObject();
    cJSON_AddStringToObject(req, "vin", properties["vin"].value<std::string>
()).c_str());
    std::string body = cJSON_PrintUnformatted(req);
    cJSON_Delete(req);
    std::map<std::string, std::string> headers = {
        {"Authorization", properties["token"].value<std::string>()},
        {"X-Token-Issuer", properties["tokenIssuer"].value<std::string>()},
        {"X-Request-Id", "1"}
    };
    std::string response = HttpPost(url, body, headers);
    return response;
});
```

2.7.6 车辆剩余保养里程接口

- MCP工具名：`car.get_maintenance_info`
- 工具描述：获取车辆剩余保养里程
- 输入参数：token、tokenIssuer、vin
- 返回：保养信息JSON
- 参考实现：

```
mcp_server.AddTool("car.get_maintenance_info", "获取车辆剩余保养里程",
PropertyList({
    Property("token", kPropertyTypeString),
    Property("tokenIssuer", kPropertyTypeString),
    Property("vin", kPropertyTypeString)
```

```

    )), [this](const PropertyList& properties) -> ReturnValue {
        std::string url =
            "https://gateway.wuzheng.com.cn/vehicle/maintenance/get_maintenance_info";
        cJSON* req = cJSON_CreateObject();
        cJSON_AddStringToObject(req, "vin", properties["vin"].value<std::string>
        (()).c_str());
        std::string body = cJSON_PrintUnformatted(req);
        cJSON_Delete(req);
        std::map<std::string, std::string> headers = {
            {"Authorization", properties["token"].value<std::string>()},
            {"X-Token-Issuer", properties["tokenIssuer"].value<std::string>()},
            {"X-Request-Id", "1"}
        };
        std::string response = HttpPost(url, body, headers);
        return response;
    });

```

2.7.7 如何自定义唤醒词

- 在 `Kconfig.projbuild` 文件中添加自定义唤醒词配置项：

```
CONFIG_CUSTOM_WAKE_WORD="你的唤醒词"
```

- 在 `custom_wake_word.cc` 文件中使用该配置项初始化唤醒词：

```

esp_mn_commands_clear();
esp_mn_command_add(CONFIG_CUSTOM_WAKE_WORD);

```

3. 功能测试方案

3.1. python实现mcp client调用小智MCP工具实现自动化测试加速开发

- 使用 Python 的 `requests` 库调用 MCP 工具接口，进行自动化测试。

```

import requests
import json
def call_mcp_tool(tool_name, properties):
    url = f"http://<device_ip>:<port>/mcp/{tool_name}"
    headers = {'Content-Type': 'application/json'}
    response = requests.post(url, headers=headers, data=json.dumps(properties))
    return response.json()
# 示例调用登录接口
login_response = call_mcp_tool("car.login", {
    "loginName": "your_username",
    "loginPwd": "your_password",
    "loginType": "1",

```

```
        "providerSys": "electric"  
    })  
    # 判断是否执行成功  
    if login_response.get("success"):  
        print("登录成功")  
    else:  
        print("登录失败")
```

4. 工作量评估

- MCP工具开发：每个接口约 2-3 小时，6 个接口约 15 小时。
- 传感器集成与驾驶行为分析：约 10 小时（含调试）。
- 语音交互与控制逻辑：约 10 小时。